

Graphes : cours

Contents

1	Introduction	4
1.1	Historique	4
1.2	Relations entre graphe et informatique	4
2	Graphes : Un modèle mathématique	4
2.1	Définitions et notations	4
2.1.1	L'objet mathématique	4
2.1.2	Vocabulaire	5
2.1.3	Propriétés de graphes	6
2.2	Notions de chemin	6
2.2.1	Définition (<i>chemin</i>)	6
2.2.2	Définition (<i>longueur d'un chemin</i>)	7
2.2.3	Définition (<i>concaténation de chemins</i>)	7
2.2.4	Définition (<i>circuit</i>)	7
2.2.5	Définition (<i>chemin élémentaire</i>)	8
2.3	Représentation de l'objet mathématique	8
2.3.1	Le graphique (diagramme sagittal)	8
2.3.2	Tableau d'attribution des arcs (ou matrice d'incidence)	8
2.3.3	Dictionnaire des successeurs	9
3	Aspect algorithmique : un type de données abstrait	9
3.1	Les sommets et les arcs	9
3.2	Trois exemples de représentation concrète	10
3.2.1	Représentation par tableaux	10
3.2.2	Représentation par tableau de listes	11
3.2.3	Représentation décentralisée par listes	11
3.3	Le type abstrait graphe : les méthodes	11
4	Relations et opérateurs	11
4.1	Relation d'ordre	12
4.2	Union	12
4.3	Composition	12
4.3.1	Définition	12
4.3.2	Proposition (<i>distributivité</i>)	12
4.3.3	Proposition (<i>compatibilité avec la relation d'ordre</i>)	13
4.3.4	Algorithme	13
4.4	Puissance d'un graphe	13

4.4.1	Définition (<i>puissance d'un graphe</i>)	13
4.4.2	Théorème	14
4.4.3	Proposition (<i>chemin et puissance</i>)	14
4.5	Graphe transposé	14
4.5.1	Définition (<i>graphe transposé</i>)	14
4.6	Sous-graphe	14
4.6.1	Définition (<i>sous-graphe</i>)	14
5	Fermeture transitive	15
5.1	Retour sur la transitivité	15
5.2	Proposition (<i>transitivité</i>)	15
5.3	Caractérisation de G^+	16
5.3.1	Proposition	16
5.3.2	Proposition	16
5.3.3	Corollaire	17
5.3.4	Proposition	17
5.4	Algorithme des puissances	18
5.5	Algorithme de ROY–WARSHALL	19
5.5.1	Définition (<i>opération θ</i>)	19
5.5.2	Nouvelle caractérisation de la fermeture transitive	20
5.5.3	Algorithme	20
5.5.4	Exemple d'exécution	21
5.6	Routage	23
5.6.1	Définition	23
5.6.2	Construction des routages par l'algorithme de ROY–WARSHALL	23
5.6.3	Exemple d'exécution	24
6	Circuits – Composantes fortement connexes	25
6.1	Détermination de l'existence de circuits	25
6.1.1	Proposition (6.1)	26
6.1.2	Exemple :	27
6.2	Composantes fortement connexes	28
6.2.1	Définition	28
6.2.2	Proposition (6.2)	29
6.2.3	Proposition (6.3)	29
6.2.4	Définition (6.2)	29
6.2.5	Définition (6.3)	29
6.2.6	Proposition (6.4)	29
6.2.7	Définition (6.4)	30
6.2.8	Proposition (6.5)	30

6.2.9	Algorithme de FOULKES	30
6.2.10	Exemple	31
7	Chemins de valeur optimale	32
7.1	Définition et problèmes posés	32
7.1.1	Définition (7.1)	32
7.1.2	Exemples	33
7.1.3	Proposition (7.1)	33
7.2	Chemins de valeur additive minimale issus d'un sommet donné (problème x -optimal)	34
7.2.1	Existence	34
7.2.2	Algorithme de DIJKSTRA	35
7.2.3	Algorithme de BELLMAN-FORD	37
7.3	Valeurs optimales de tout sommet à tout sommet	40
7.3.1	Lemme 1	40
7.3.2	Lemme 2	40
7.3.3	Lemme 3	40
7.3.4	Rappel de l'algorithme de ROY-WARSHALL avec routage	40
7.3.5	Algorithme de FLOYD-WARSHALL	41

List of Algorithms

1	Représentation de graphe par tableau	10
2	Calcul de $G_3 = G_1 \circ G_2$	13
3	Progression	18
4	Initialisation	18
5	Algorithme de la fermeture transitive par calcul des puissances	19
6	ROY-WARSHALL	20
7	ROY-WARSHALL modifié	24
8	Algorithme de MARIMONT	27
9	Algorithme de FOULKES	31
10	DIJKSTRA	36
11	BELLMAN-FORD, première version	38
12	BELLMAN-FORD, seconde version	38
13	Détection de circuit absorbant	39
14	Rappel de l'algorithme de ROY-WARSHALL avec routage	41
15	Algorithme de FLOYD-WARSHALL	41

1 Introduction

1.1 Historique

1736 : EULER : problème des ponts de KÖNIGSBERG.

Jusqu'en 1946 : domaine des mathématiques.

19ème siècle : KIRCHHOFF, HAMILTON, SYLVESTER, ...

20ème siècle : POINCARÉ, KURATOWSKI, HALL, ...

Deuxième guerre mondiale : naissance de la Recherche Opérationnelle.

Théorie des graphes pour modéliser des problèmes.

DANTZIG, FORD, FULBERSON, ...

Troisième époque : années 60, sciences de l'information.

Algorithmique des graphes.

DIJKSTRA, KNUTH, WIRTH, ...

1.2 Relations entre graphe et informatique

À double sens :

- (1) l'informatique, outil de résolution de problèmes de graphes ;
- (2) les graphes, outils de modélisation de problèmes informatiques.

En ce qui concerne (2) : il existe de nombreux problèmes informatiques où les problèmes se modélisent à l'aide de graphes.

Par exemple :

- Réseaux, systèmes ;
- Bases de données (RDF) ;
- Représentation des connaissances (IA) : graphes conceptuels ;
- ...

2 Graphes : Un modèle mathématique

2.1 Définitions et notations

2.1.1 L'objet mathématique

- Vision ensembliste :

Définition : un graphe est un doublet $G = (X, \Gamma)$, où :

- X est un ensemble (en général fini) appelé *ensemble des sommets* ;
- Γ est un ensemble de couples $(x, y) \in X^2$, appelé *ensemble des arcs*.

Exemple :

Soit $X = \llbracket 1 ; 9 \rrbracket$, et

$$\Gamma = \{(1, 2), (1, 3), (2, 5), (3, 3), (3, 4), (3, 6), (4, 2), (4, 5), (5, 6), (5, 7), (5, 8), (6, 3), (8, 7)\}$$

• Vision fonctionnelle :

Définition : un graphe est un doublet $G = (X, \Gamma)$, où :

- X est un ensemble de sommets ;
- Γ est une fonction de X dans l'ensemble $\mathcal{P}(X)$ des parties de X (aussi noté 2^X).

Ainsi à tout sommet x est associé l'ensemble $\Gamma(x) \subseteq X$.

Exemple :

Soit $X = \llbracket 1 ; 9 \rrbracket$, et :

$$\begin{aligned} \Gamma(1) &= \{2, 3\} \\ \Gamma(2) &= \{5\} \\ \Gamma(3) &= \{3, 4, 6\} \\ \Gamma(4) &= \{2, 5\} \\ \Gamma(5) &= \{6, 7, 8\} \\ \Gamma(6) &= \{3\} \\ \Gamma(7) &= \emptyset \\ \Gamma(8) &= \{7\} \\ \Gamma(9) &= \emptyset \end{aligned}$$

• Vision relationnelle :

À tout graphe $G = (X, \Gamma)$ est associé une relation binaire $\mathcal{R}$ sur X , et réciproquement, par l'équivalence suivante :

$$x\mathcal{R}y \Leftrightarrow (x, y) \in \Gamma$$

2.1.2 Vocabulaire

Pour un arc $(x, y) \in \Gamma$, on dira que x est *l'origine*, et y est *l'extrémité*.

On dit aussi que y est *successeur* de x , et que x est *prédécesseur* de y .

On note

$$\Gamma(x) = \{y \in X \mid (x, y) \in \Gamma\}$$

l'ensemble des successeurs de x .

On note

$$\Gamma^{-1}(y) = \{x \in X \mid (x, y) \in \Gamma\}$$

l'ensemble des prédécesseurs de y .

Pour $A \subset X$, on note

$$\Gamma(A) = \bigcup_{x \in A} \Gamma(x)$$

et

$$\Gamma^{-1}(A) = \bigcup_{x \in A} \Gamma^{-1}(x)$$

On note

$$d^+(x) = \text{card}(\Gamma(x))$$

le *demi degré extérieur* de x .

On note

$$d^-(x) = \text{card}(\Gamma^{-1}(x))$$

le *demi degré intérieur* de x .

On note $d(x) = d^+(x) + d^-(x)$ le *degré* de x .

2.1.3 Propriétés de graphes

Propriétés que peuvent avoir certains graphes (hors transitivité, on la verra plus loin).

Dans la suite, on note $\Delta = \{(x, x) \mid x \in X\}$, *i.e* l'ensemble des boucles (arc qui relie un sommet à lui-même).

- Réflexivité : un graphe $G = (X, \Gamma)$ est *réflexif* si et seulement si

$$\forall x \in X, (x, x) \in \Gamma \Leftrightarrow \Delta \subseteq \Gamma$$

- Anti-réflexivité : un graphe $G = (X, \Gamma)$ est *anti-réflexif* si et seulement si

$$\forall x \in X, (x, x) \notin \Gamma \Leftrightarrow \Delta \cap \Gamma = \emptyset$$

- Symétrie : un graphe $G = (X, \Gamma)$ est *symétrique* si et seulement si

$$\forall (x, y) \in X^2, (x, y) \in \Gamma \Rightarrow (y, x) \in \Gamma$$

Ce qui est équivalent à $\Gamma = \Gamma^{-1}$.

- Antisymétrie : un graphe $G = (X, \Gamma)$ est dit *antisymétrique* si et seulement si

$$\forall (x, y) \in X^2, \begin{cases} (x, y) \in \Gamma \\ (y, x) \in \Gamma \end{cases} \Rightarrow x = y$$

Ce qui est équivalent à $\Gamma \cap \Gamma^{-1} \subset \Delta$.

2.2 Notions de chemin

2.2.1 Définition (*chemin*)

Un chemin de G est une suite de sommets $[x_{i_1}, x_{i_2}, \dots, x_{i_k}]$ telle que

$$\forall j \in \llbracket 1 ; k - 1 \rrbracket, (x_{i_j}, x_{i_{j+1}}) \in \Gamma$$

x_{i_1} est *l'origine* du chemin ;

x_{i_k} est *l'extrémité* du chemin ;

Les x_{i_j} (pour $j \in \llbracket 2 ; k - 1 \rrbracket$), s'ils existent, sont les *sommets intermédiaires*.

Un chemin de x à y dont on ne précise pas les sommets intermédiaires sera noté :

$$u = [x * y]$$

On dira que y est un *descendant* de x , et on dira que x est un *ascendant*, ou *ancêtre* de y .

Si l'on souhaite désigner des sommets intermédiaires, pas pas tous, alors on utilisera la notation suivante :

$$u = [x_1 * x_p * x_q * x_k]$$

(u est un chemin de x_1 à x_k qui passe par x_p et par x_q).

2.2.2 Définition (*longueur d'un chemin*)

Soit u un chemin dans un graphe.

La longueur du chemin u , notée $|u|$, est le nombre d'arcs qui le composent.

2.2.3 Définition (*concaténation de chemins*)

Soient u, v des chemins dans un graphe tels que l'extrémité de u coïncide avec l'origine de v .

On note $U \cdot v$ la *concaténation* des chemins u et v .

On a alors

$$|u \cdot v| = |u| + |v|$$

2.2.4 Définition (*circuit*)

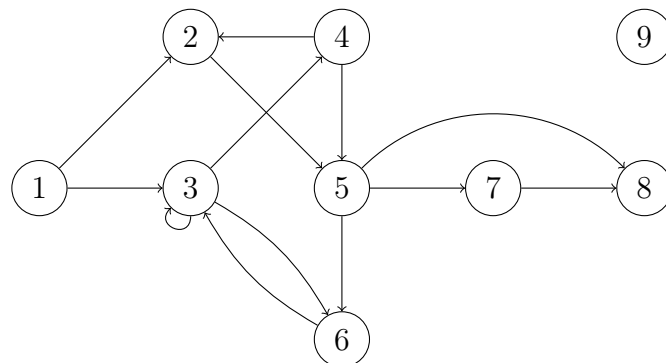
Un *circuit* est un chemin dont l'extrémité coïncide avec l'origine.

2.2.5 Définition (*chemin élémentaire*)

Un chemin (resp. circuit) *élémentaire* est défini par une suite de sommets *sans répétition* (sauf l'origine et l'extrémité dans le cas d'un circuit).

2.3 Représentation de l'objet mathématique

2.3.1 Le graphique (diagramme sagittal)



Ce type de représentation pose des problèmes de topologie (croisements d'arcs) et de lisibilité quand il y a beaucoup de sommets et / ou d'arcs.

Elle n'est adaptée que pour les "petits" graphes ($\text{card}(X) \leq 10$).

2.3.2 Tableau d'attribution des arcs (ou matrice d'incidence)

Cela correspond à une matrice booléenne.

	1	2	3	4	5	6	7	8	9
1		1	1						
2					1				
3			1	1		1			
4		1			1				
5						1	1	1	
6			1						
7									
8							1		
9									

2.3.3 Dictionnaire des successeurs

1	2, 3
2	5
3	3, 4, 6
4	2, 5
5	5, 7, 8
6	3
7	
8	7
9	

Resp. des prédécesseurs :

1	
2	1, 4
3	1, 3, 6
4	3
5	2, 4
6	3, 5
7	5, 8
8	5
9	

3 Aspect algorithmique : un type de données abstrait

Problème : représenter, à l'aide de structures de données algorithmiques, le doublet mathématique (X, Γ) , avec $\Gamma \subseteq X^2$.

3.1 Les sommets et les arcs

- Le type sommet

On suppose que tout sommet est identifié par un numéro (un entier), que l'on désignera par `num` (en général, on suppose `num` ≥ 1).

Notation algorithmique : `ENT SOMMET :: NUM`

Les objets de type sommet ont un attribut de nom `num` et de type entier.

L'expression `x.num` désigne le numéro du sommet x .

- Le type arc :

`SOMMET ARC :: ori ;`

`SOMMET ARC :: ext ;`

Si γ est un objet de type `arc`, l'expression $\gamma.\text{ori}$ (resp. $\gamma.\text{ext}$) désigne le sommet origine (resp. extrémité) de l'arc γ , et $\gamma.\text{ori.num}$ (resp. $\gamma.\text{ext.num}$) le numéro du sommet origine (resp. extrémité) de γ .

Par commodité, on désignera souvent par (x, y) l'arc γ tel que $\gamma.\text{ori} = x$ et $\gamma.\text{ext} = y$.
Ou par (i, j) l'arc γ tel que $\gamma.\text{ori.num} = i$ et $\gamma.\text{ext.num} = j$.

3.2 Trois exemples de représentation concrète

On suppose connue les structures de données **TABLEAU**[G] (tableau à une dimension), **TABLEAU2**[G] (tableau à deux dimensions), et **LISTE**[G] (liste chaînée), où G désigne le type des éléments contenus dans ces structures.

3.2.1 Représentation par tableaux

Algorithm 1: Représentation de graphe par tableau

```

1 ENT GRAPHE :: nmax // nombre max de sommets
2 TABLEAU[BOOL] GRAPHE :: vs // G.vs[i] indique l'existence du
  sommet i ( $i \in \llbracket 1 ; nmax \rrbracket$ )
3 TABLEAU2[BOOL] GRAPHE :: m // G.m[i, j] indique l'existence de
  l'arc (i, j) ( $i, j \in \llbracket 1 ; nmax \rrbracket$ )

```

On a toujours l'implication

$$\forall i, j, G.m[i, j] \Rightarrow G.vs[i] \wedge G.vs[j]$$

(un arc ne peut relier que des sommets existants)

Dans cette représentation de $G = (X, \Gamma)$, les ensembles X et Γ sont représentés de la manière suivante :

$$x \in X \Leftrightarrow G.vs[x.num]$$

$$(x, y) \in \Gamma \Leftrightarrow G.m[x.num, y.num]$$

- Exemple : reprenons le graphe précédant.

$nmax = 10$

vs :

V	V	V	V	V	V	V	V	V	F
---	---	---	---	---	---	---	---	---	---

m :

	1	2	3	4	5	6	7	8	9
1		V	V						
2					V				
3			V	V		V			
4		V			V				
5						V	V	V	
6			V						
7									
8							V		
9									

3.2.2 Représentation par tableau de listes

Tableau[Liste[Sommets]] Graphe::ts;
 $G.ts[i]$ = la liste des successeurs du sommet n° i .
 $x \in G \Leftrightarrow G.vs[x.num]$.

• Exemple :

1	→	2, 3
2	→	5
3	→	3, 4, 6
4	→	2, 5
5	→	6, 7, 8
6	→	3
7	→	
8	→	7
9	→	
10	→	

3.2.3 Représentation décentralisée par listes

On enrichit le type `Sommet` : chaque sommet a maintenant comme attribut la liste de ses successeurs.

```
ENT Sommet :: num;
Liste[Sommet] Sommet :: ls ;
Liste[Sommet] Graphe :: lson;
```

On a $x \in X \Leftrightarrow G.lson$ contient x
et $(x, y) \in \Gamma \Leftrightarrow G.lson$ contient x et $x.ls$ contient y .

3.3 Le type abstrait graphe : les méthodes

On supposera disponibles les méthodes suivantes :

- `créer` : procédure qui initialise le graphe courant au graphe vide ;
- `ajout_sommet(int i)` : ajout au graphe du sommet de numéro i ;
- `ajout_arc(int i, j)` : ajout au graphe de l'arc (i, j) (et au besoin des sommets i et j).

4 Relations et opérateurs

On désignera par $\mathcal{G}(X)$ l'ensemble des graphes ayant X comme ensemble de sommets.

On note

$$G_1 = (X, \Gamma_1) \quad G_2 = (X, \Gamma_2)$$

deux éléments de $\mathcal{G}(X)$.

4.1 Relation d'ordre

La relation

$$G_1 \preceq G_2 \Leftrightarrow \Gamma_1 \subset \Gamma_2$$

se lit " G_1 est un *graphe partiel* de G_2 ", et est une relation d'ordre partiel sur $\mathcal{G}(X)$.

Pour cette relation,

- $\mathcal{G}(X)$ a un minimum $(X, \emptyset)$;
- $\mathcal{G}(X)$ a un maximum (X, X^2) .

4.2 Union

Définition :

$$G_1 \cup G_2 = (X, \Gamma_1 \cup \Gamma_2)$$

4.3 Composition

4.3.1 Définition

Composition de G_1 et G_2 :

$$G_3 = G_1 \circ G_2 = (X, \Gamma_3)$$

où

$$\Gamma_3 = \left\{ (x, y) \mid \exists z \in X \left| \begin{array}{l} (x, z) \in \Gamma_1 \\ (z, y) \in \Gamma_2 \end{array} \right. \right\}$$

On remarque que l'opérateur $\circ$ n'est pas commutatif.

Mais il est associatif, et possède un élément neutre : le graphe "diagonal" $D = (X, \Delta)$, où

$$\Delta = \{(x, x) \mid x \in X\}$$

En effet, $\forall G \in \mathcal{G}(X)$, $G \circ D = D \circ G = G$.

4.3.2 Proposition (*distributivité*)

L'opérateur $\circ$ est distributif par rapport à l'union.

Autrement dit, $\forall G_1, G_2, G_3 \in \mathcal{G}(X)$, on a :

$$(G_1 \cup G_2) \circ G_3 = (G_1 \circ G_3) \cup (G_2 \circ G_3)$$

$$G_1 \circ (G_2 \cup G_3) = (G_1 \circ G_2) \cup (G_1 \circ G_3)$$

Par contre, l'union n'est pas distributive sur la composition.

4.3.3 Proposition (*compatibilité avec la relation d'ordre*)

L'opérateur $\circ$ est compatible à gauche et à droite avec la relation d'ordre définie précédemment :

$$\forall (G_k)_{k \in \llbracket 1 ; 3 \rrbracket} \subset \mathcal{G}(X),$$

$$G_1 \preceq G_2 \Rightarrow \begin{cases} (G_1 \circ G_3) \preceq (G_2 \circ G_3) \\ (G_3 \circ G_1) \preceq (G_3 \circ G_2) \end{cases}$$

4.3.4 Algorithme

Calcul de $G_3 = G_1 \circ G_2$ en supposant que $X = \llbracket 1 ; n \rrbracket$.

Algorithm 2: Calcul de $G_3 = G_1 \circ G_2$

```

1 Function composer(graphe  $G_1, G_2$ ) : graphe:
2   var sommet  $x, y, z$  ;
3   var bool  $b$  ;
4   var graphe  $res$  ;
5   for tout  $x$  de  $G_1.ens\_som$  do
6     for tout  $y$  de  $G_2.ens\_som$  do
7        $z \leftarrow 0$ ;
8        $b \leftarrow \text{false}$ ;
9       while  $z < n$  et non  $b$  do
10         $z \leftarrow z + 1$ ;
11         $b \leftarrow G_1.valid\_arc(x, z)$  et  $G_2.valid\_arc(z, y)$ ;
12      if  $b$  then
13         $res.ajout\_arc(x, y)$ ;
14   renvoyer  $res$ ;

```

Complexité : $\mathcal{O}(n^3)$.

4.4 Puissance d'un graphe

4.4.1 Définition (*puissance d'un graphe*)

On définit par récurrence sur $p \in \mathbb{N}$ le graphe

$$G^{[p]} = (X, \Gamma^{[p]})$$

de la façon suivante :

$$\begin{cases} G^{[0]} = D \\ \forall p \in \mathbb{N}^*, G^{[p]} = G^{[p-1]} \circ G = G \circ G^{[p-1]} \end{cases}$$

4.4.2 Théorème

On a :

$$G^{[k]} \circ G^{[l]} = G^{[k+l]}$$

4.4.3 Proposition (*chemin et puissance*)

Soit $G = (X, \Gamma)$ un graphe.

Alors $(x, y) \in \Gamma^{[p]} \Leftrightarrow \exists$ un chemin de longueur p dans G allant de x à y .

La démonstration est triviale.

4.5 Graphe transposé

4.5.1 Définition (*graphe transposé*)

Le graphe transposé de $G = (X, \Gamma)$ est

$$G^t = (X, \Gamma^t)$$

avec

$$(x, y) \in \Gamma^t \Leftrightarrow (y, x) \in \Gamma$$

C'est le graphe déduit de G par inversion du sens de tous les arcs.

4.6 Sous-graphe

4.6.1 Définition (*sous-graphe*)

Soit $G = (X, \Gamma) \in \mathcal{G}(X)$, et $A \subset X$.

Le *sous-graphe engendré par A* est défini par :

$$G_A = (A, \Gamma_A) \in \mathcal{G}(A)$$

avec

$$\Gamma_A = \left\{ (x, y) \in \Gamma \mid \begin{array}{l} x \in A \\ y \in A \end{array} \right\}$$

5 Fermeture transitive

Problème : étant donné deux sommets x et y d'un graphe $G = (X, \Gamma)$, existe-t-il un chemin de x à y ?

On va construire le graphe $G^+ = (X, \Gamma^+)$ tel que

$$(x, y) \in \Gamma^+ \Leftrightarrow \exists \text{ un chemin de } x \text{ à } y \text{ dans } G$$

On appelle G^+ la *fermeture transitive* de G .

5.1 Retour sur la transitivité

Un graphe est transitif si et seulement si la relation binaire associée est transitive :

$$\forall (x, y, z) \in X^3, \begin{cases} (x, y) \in \Gamma \\ (y, z) \in \Gamma \end{cases} \Rightarrow (x, z) \in \Gamma$$

5.2 Proposition (*transitivité*)

Un graphe G est transitif si et seulement si tout chemin de longueur 2 est sous-tendu par un arc, *i.e*

$$\Gamma^{[2]} \subseteq \Gamma$$

□ Preuve :

$$\begin{aligned} G \text{ transitif} &\Rightarrow \forall x, y, z \in X, \begin{cases} (x, y) \in \Gamma \\ (y, z) \in \Gamma \end{cases} \Rightarrow (x, z) \in \Gamma \\ &\Rightarrow \forall x, z, \left(\exists y \mid \begin{cases} (x, y) \in \Gamma \\ (y, z) \in \Gamma \end{cases} \Rightarrow (x, z) \in \Gamma \right) \\ &\dots\dots\dots \end{aligned}$$

- Réciproquement, supposons $\Gamma^{[2]} \subseteq \Gamma$.

Alors :

$$\begin{aligned} \begin{cases} (x, z) \in \Gamma \\ (z, y) \in \Gamma \end{cases} &\Rightarrow (x, y) \in \Gamma^{[2]} \\ &\Rightarrow (x, y) \in \Gamma \\ &\Rightarrow G \text{ transitif} \end{aligned}$$

■

Corollaire évident :

Un graphe G est transitif si et seulement si tout chemin est sous-tendu par un arc,
i.e

$$\forall k \in \mathbb{N}^* \setminus \{1\}, \Gamma^{[k]} \subseteq \Gamma$$

Preuve par récurrence sur k .

Interprétation de ce corollaire : dans un graphe transitif, la notion de chemin se confond avec la notion d'arc.

Algorithme de calcul de la fermeture transitive ?

Pourquoi parle-t-on de *fermeture transitive* ?

Parce que :

- G^+ est transitif (par construction !) ;
- G^+ est le plus petit graphe transitif $\succcurlyeq G$

Plus précisément :

- G^+ est transitif ;
- $G \preccurlyeq G^+$;
- Si H est transitif et $G \preccurlyeq H$, alors $G^+ \preccurlyeq H$.

Intuitivement, on obtient G^+ à partir de G en ajoutant "juste ce qu'il faut" d'arcs pour rendre le graphe transitif.

5.3 Caractérisation de G^+

5.3.1 Proposition

On a :

$$G^+ = \bigcup_{k \in \mathbb{N}^*} G^{[k]}$$

□ Preuve :

Posons $G^+ = (X, \Gamma^+)$, et $\forall k \in \mathbb{N}^*, G^{[k]} = (X, \Gamma^{[k]})$.

Par définition,

$$\begin{aligned} (x, y) \in \Gamma^+ &\Leftrightarrow \exists u = [x * y] \in G \\ &\Leftrightarrow (x, y) \in \Gamma^{[k]} \quad (k = |u|) \\ &\Leftrightarrow (x, y) \in \bigcup_{k \in \mathbb{N}^*} \Gamma^{[k]} \end{aligned}$$

■

5.3.2 Proposition

Cette proposition montre que le problème se réduit à l'existence de chemins élémentaires.

Soit G un graphe.

Les deux énoncés suivants sont équivalents :

- (i) Dans G , il existe un chemin de x à y ;
- (ii) Dans G , il existe un chemin élémentaire de x à y .

□ Preuve :

- $(ii) \Rightarrow (i)$ évident puisque tout chemin élémentaire est un chemin.

- $(i) \Rightarrow (ii)$ Supposons qu'il existe un chemin $u = [x * y]$.

S'il est élémentaire, la preuve est terminée.

Sinon, la suite de sommets le définissant compte au moins une répétition.

Le chemin est donc de la forme suivante :

$$u = [x * z * z * y]$$

(avec éventuellement $z \in \{x, y\}$).

On a donc $u = u_1 \cdot u_2 \cdot u_3$, avec

$$\begin{aligned} u_1 &= [x * z] \\ u_2 &= [z * z] \\ u_3 &= [z * y] \end{aligned}$$

On pose alors $u' = u_1 \cdot u_3$. C'est un chemin de x à y obtenu en "court-circuitant" le circuit $u_2 = [z * z]$.

En itérant ce raisonnement tant qu'il y a des répétitions (sur toutes les répétitions) (des circuits), on obtient nécessairement un chemin élémentaire de x à y . ■

5.3.3 Corollaire

Des deux propositions précédentes, découle le corollaire :

Soient $G = (X, \Gamma)$ et $n = |X|$.

On a alors :

$$G^+ = \bigcup_{k=1}^n G^{[k]}$$

5.3.4 Proposition

Soit k la longueur max des chemins élémentaires de G .

Alors :

$$(1) \quad k \leq n$$

(2)

$$\bigcup_{j=1}^k G^{[j]} = \bigcup_{j=1}^{k+1} G^{[j]}$$

5.4 Algorithme des puissances

Basé sur la proposition 5.3.4.

On maintient invariante la proposition suivante :

$$\left(G^+ = \bigcup_{i=1}^k G^{[i]} \right) \wedge \left(H = \bigcup_{i=1}^{k-1} G^{[i]} \right) \wedge k \in \llbracket 1 ; n \rrbracket$$

Le calcul sera terminé lorsque $H = G^+$.

Progression : mettre à jour G^+ et H :

Algorithm 3: Progression

- 1 $H \leftarrow G^+$;
 - 2 $G^+ \leftarrow G \cup (G^+ \circ G)$;
-

En effet :

Si $G^+ = \bigcup_{j=1}^k G^{[j]}$, alors

$$G^+ \circ G = \bigcup_{j=2}^{k+1} G^{[j]}$$

(distributivité de $\circ$ sur $\cup$), d'où :

$$G \cup (G^+ \circ G) = \bigcup_{j=1}^{k+1} G^{[j]}$$

- Initialisation :

Algorithm 4: Initialisation

- 1 $G^+ \leftarrow G$;
 - 2 H arbitraire (mais différent de G^+ , par exemple $H \leftarrow (\emptyset, \emptyset)$);
-

- Algorithme :

Algorithm 5: Algorithme de la fermeture transitive par calcul des puissances

```

1 Function fermeture_transitive_puissance(graphe  $G$ ) : graphe:
2   var graphe,  $H$ , res;
3    $res \leftarrow G$ ;
4    $H$ .créer;
5   while  $H \neq res$  do
6      $H \leftarrow res$ ;
7      $res \leftarrow G \cup (G \cup res)$ ;
8   renvoyer res;

```

Complexité :

Dans le pire cas, on exécute n pas d'itérations.

À chaque pas, sont effectués :

- deux affectations de graphe (en $\mathcal{O}(n^2)$ accès élémentaires) ;
- une comparaison de deux graphes (en $\mathcal{O}(n^2)$ accès élémentaires) ;
- une opération $\circ$ (en $\mathcal{O}(n^3)$ accès élémentaires) ;
- une opération $\cup$ (en $\mathcal{O}(n^2)$ accès élémentaires).

Cet algorithme est donc en $\mathcal{O}(n^4)$ accès élémentaires.

De plus, l'espace utilisé est de trois graphes.

5.5 Algorithme de ROY–WARSHALL

- Avantages :
 - Sa complexité est en $\mathcal{O}(n^3)$;
 - L'espace utilisé est de un graphe ;
 - Il offre la possibilité de calculer des routages, *i.e* d'obtenir le tracé de chemins élémentaires.
- Principe : l'algorithme est basé sur l'opération θ .

5.5.1 Définition (*opération* θ)

Soit $x \in X$.

Si $G = (X, \Gamma) \in \mathcal{G}(X)$, alors $\theta_x(\Gamma) \in \mathcal{G}(X)$ a pour ensemble d'arcs :

$$\theta_x(\Gamma) = \Gamma \cup \left\{ (y, z) \mid \begin{array}{l} (y, x) \in \Gamma \\ (x, z) \in \Gamma \end{array} \right\}$$

Autrement dit, θ_x enrichit le graphe en lui ajoutant les arcs reliant les prédécesseurs de x aux successeurs de x .

5.5.2 Nouvelle caractérisation de la fermeture transitive

L'algorithme est basé sur une nouvelle caractérisation de la fermeture transitive :

$$G^+ = \prod_{x \in X} \theta_x(G) \quad (1)$$

où $\prod$ désigne le produit (composition) des opérations θ_x , *i.e.*, si $X = \llbracket 1 ; n \rrbracket$ est une numérotation des sommets, alors

$$G^+ = \theta_n(\theta_{n-1}(\cdots(\theta_1(G))\cdots))$$

La caractérisation (1) est basée sur le lemme suivant :

Soit V_i l'ensemble des couples $(y, z) \in X^2$ tels qu'il existe dans G entre y et z un chemin élémentaire de longueur 1 ou dont tous les sommets intermédiaires sont de numéro $\leq i$.

Alors

$$\forall i \in \llbracket 1 ; n \rrbracket, \theta_i(\theta_{i-1}(\cdots(\theta_1(G))\cdots)) = V_i$$

Comme tout chemin de G est de longueur 1 ou possède des sommets intermédiaires dans $\llbracket 1 ; n \rrbracket$ le caractérisant valide.

5.5.3 Algorithme

Algorithm 6: ROY-WARSHALL

```

1 Function roy-warshall(graphe  $G$ ) : graphe:
2   var sommet, x, y, z;
3   var ens[sommet]  $X$ ;
4   res  $\leftarrow G$ ;
5    $X \leftarrow G.lst\_som$ ;
6   for tout  $x \in X$  do
7     for tout  $y \in X$  do
8       if res.validarc( $y, x$ ) then
9         //  $y$  est un prédécesseur de  $x$ 
10        for tout  $z \in X$  do
11          if res.validarc( $x, z$ ) then
12            //  $z$  est un successeur de  $x$ 
13            res.ajouter_arc( $y, z$ );

```

- Complexité :

Pour chaque opération θ , il y a au plus :

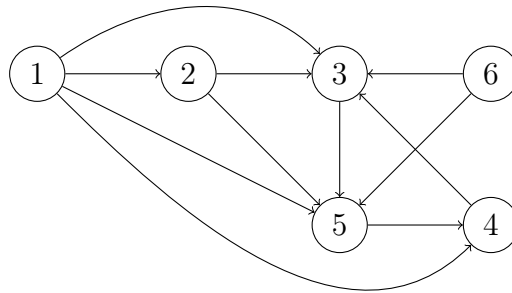
- $n + n^2$ accès à **validarc**
- n^2 accès à **ajouter_arc**

Soit globalement :

au plus $n^2 + n^3$ accès `validarc`, n^3 accès à `ajouter_arc`.
L'algorithme est donc bien en $\mathcal{O}(n^3)$.

5.5.4 Exemple d'exécution

Soit le graphe suivant :



Que l'on représente par matrice booléenne :

	1	2	3	4	5	6
1		×	×	×	×	
2			×		×	
3					×	
4			×			
5				×		
6			×		×	

Application de θ_i :

Pour chaque croix de la colonne i , on "recopie" dans cette ligne la ligne i .

θ_1

	1	2	3	4	5	6
1		×	×	×	×	
2			×		×	
3					×	
4			×			
5				×		
6			×		×	

Il ne se passe rien car 1 n'a pas de prédécesseur.

θ_2

	1	2	3	4	5	6
1		×	×	×	×	
2			×		×	
3					×	
4			×			
5				×		
6			×		×	

Il ne se passe rien non plus car les successeurs de 2 sont déjà des successeurs de 1.

θ_3

	1	2	3	4	5	6
1		×	×	×	×	
2			×		×	
3					×	
4			×		×	
5				×		
6			×		×	

θ_4

	1	2	3	4	5	6
1		×	×	×	×	
2			×		×	
3					×	
4			×		×	
5			×	×	×	
6			×		×	

θ_5

	1	2	3	4	5	6
1		×	×	×	×	
2			×	×	×	
3			×	×	×	
4			×	×	×	
5				×	×	
6			×	×	×	

θ_6

	1	2	3	4	5	6
1		×	×	×	×	
2			×	×	×	
3			×	×	×	
4			×	×	×	
5				×	×	
6			×	×	×	

Il ne se passe rien.

5.6 Routage

5.6.1 Définition

Idée : être capable de répondre à la question $(\forall x, y \in X)$:

Existe-il un chemin de x à y , et si oui, fournir un tel chemin.

Solution : fondée sur la notion de *table*.

Le type abstrait $\text{TABLE}[K, E]$ décrit l'ensemble des tables dont l'ensemble des clés est K et l'ensemble des valeurs est E .

Une table est manipulée *via* les opérations suivantes :

- $T[x]$ délivre la valeur v de type E correspondant à la clé x (de type K) ;
- $T[x] \leftarrow v$ affecte la valeur v à l'élément correspondant à la clé x .

Les objets de type **SOMMET** sont alors munis d'une table :

$\text{TABLE}[\text{SOMMET}, \text{SOMMET}] \text{ SOMMET} :: \text{R};$

Signification : pour tous sommets y, z :

$$y.\text{R}[z] = \begin{cases} \text{nil} & \text{si } z \notin \Gamma^+(y) \\ \text{successeur de } y \text{ sur un chemin élém de } y \text{ à } z \text{ dans } G & \text{sinon} \end{cases}$$

(**nil** signifie "pas de successeur").

5.6.2 Construction des routages par l'algorithme de ROY-WARSHALL

On considère, $\forall y \in X$, les valeurs initiales de la table $y.\text{R}$:

$$\forall z \in X, y.\text{R}[z] = \begin{cases} \text{nil} & \text{si } (y, z) \notin \Gamma \\ z & \text{sinon} \end{cases}$$

(on se restreint aux chemins de longueur 1).

On modifie l'opération θ_x pour qu'elle mette à jour les valeurs des tables **R** dans le cas où θ_x ajoute un arc entre y et z .

Alors $y.\text{R}[z] \leftarrow y.\text{R}[x]$.

(pour aller de y à z , on doit d'abord aller à x).

Algorithm 7: ROY-WARSHALL modifié

```

1 Function roy-warshall-modif(graphe G) : graphe:
2   var sommet x, y, z;
3   var ens[sommet] X;
4   graphe res;
5   res ← G;
6   X ← G.lst_nom;
7   for tout y ∈ X do
8     for tout z ∈ X do
9       if validarc(y, z) then
10        | y.R[z] ← z;
11      else
12        | y.R[z] ← nil;
13  for tout x ∈ X do
14    for tout y ∈ X do
15      if res.validarc(y, x) then
16        for tout z ∈ X do
17          if res.validarc(x, z) then
18            if non validarc(y, z) then
19              res.ajouter_arc(y, z);
20              y.R[z] ← y.R[x];
21  renvoyer res;

```

5.6.3 Exemple d'exécution

On utilise une matrice **R** telle que

$$\forall y, z, \text{ R}[y, z] = \begin{cases} y.\text{R}[z] & \text{si } (y, z) \in \Gamma \\ 0 & \text{sinon} \end{cases}$$

	1	2	3	4	5	6	7	8	9
1						6			
2									
3		2					7	8	
4	1				5				
5							7		
6				4					
7								8	
8			3						
9	1	2							

- Opération θ_1 : on parcourt la colonne et on s'arrête sur chaque ligne de valeur non nulle (correspondant à `res.validarc(y, 1)` pour une telle ligne, on effectue la recopie des éléments non nuls de la ligne i en toute case nulle $R[y, z]$ telle que $R[1, z] \neq 0$).

Ici : ne modifie que les lignes 4 et 9.

	1	2	3	4	5	6	7	8	9
1						6			
2									
3		2					7	8	
4	1				5	1			
5							7		
6				4					
7								8	
8			3						
9	1	2				1			

θ_2 n'ajoute aucun arc (2 n'a pas de successeur).

De même pour θ_9 , car 9 n'a pas de prédécesseurs.

Résultat final :

	1	2	3	4	5	6	7	8	9
1	6	6	6	6	6	6	6	6	
2									
3		2	8				7	8	
4	1	5	5	1	5	1	5	5	
5		7	7				7	7	
6	4	4	4	4	4	4	4	4	
7		8	8				8	8	
8		3	3				3	3	
9	1	2	1	1	1	1	1	1	

Tracé d'un chemin du sommet 1 au sommet 2 :

$R[1, 2] = 6$; $R[6, 2] = 4$;

$R[4, 2] = 5$; $R[5, 2] = 7$;

$R[7, 2] = 8$; $R[8, 2] = 3$; $R[3, 2] = 2$

D'où le chemin : $[1, 6, 4, 5, 7, 8, 3, 2]$.

6 Circuits – Composantes fortement connexes

6.1 Détermination de l'existence de circuits

- Applications : réseaux (interblocage), ordonnancement de tâches (*cf* TD).
- Première idée : appliquer l'algorithme de ROY–WARSHALL (fermeture transitive).

Un sommet x situé sur un circuit vérifie $(x, x) \in G^+$.

Problème : complexité élevée.

- Seconde idée : concevoir un algorithme spécifique : algorithme de MARIMONT.

Il est fondé sur la proposition suivante :

6.1.1 Proposition (6.1)

Les énoncés suivants sont équivalents :

- (1) G est sans circuit ;
- (2) G et tous ses sous-graphes possèdent au moins un point d'entrée (sommet sans prédécesseur) et un point de sortie (sommet sans successeur).

□ Preuve :

Supposons $G = (X, \Gamma)$ sans circuit, et que G n'a pas de point de sortie.

Alors $\forall x \in X, \exists y \in X \mid y \in \Gamma(x)$.

Donc partant d'un sommet x_i quelconque, on peut construire un chemin $x_i x_{i_1} \cdots x_{i_k}$ de longueur aussi grande que l'on veut.

Dès que sa longueur est supérieure à $n = |X|$, on obtient un circuit, ce qui entre en contradiction avec l'hypothèse.

On peut faire de même pour les points d'entrée (en construisant le chemin dans le sens inverse).

Enfin, si G est sans circuit, alors tout sous-graphe engendré par une partie de G est également sans circuit.

On a donc démontré $(1) \Rightarrow (2)$.

- $(2) \Rightarrow (1)$ par contraposée.

Supposons que G ait un circuit γ , et soit G_γ le sous-graphe engendré par les sommets du circuit γ .

Par construction, G_γ n'a ni point d'entrée, ni point de sortie. ■

Le principe de l'algorithme est le suivant :

Si G ne possède pas de point d'entrée ou de points de sortie, on peut conclure que G a au moins un circuit.

Sinon, on ôte les points d'entrée et de sortie, et on recommence sur le sous-graphe engendré de la sorte.

Si on aboutit au graphe vide, on conclut que G était sans circuit.

Remarque : on peut ne considérer que les points d'entrée, ou que les points de sortie, ou les deux.

Algorithm 8: Algorithme de MARIMONT

```

1 Function Marimont(graphe G) : bool:
2   var graphe H; ens[sommet] E, S; bool hvide;
3    $H \leftarrow G$ ;
4    $hvide \leftarrow H.vide$ ;
5   if non hvide then
6      $E \leftarrow H.points\_entrée$ ;
7      $S \leftarrow H.points\_sortie$ ;
8   while non hvide et  $E \neq \emptyset$  et  $S \neq \emptyset$  do
9      $H \leftarrow H.sousgraphe(G.lst\_som \setminus (E \cup S))$ ;
10     $hvide \leftarrow H.vide$ ;
11    if non hvide then
12       $E \leftarrow H.points\_entrée$ ;
13       $S \leftarrow H.points\_sortie$ ;
14  renvoyer  $\neg hvide$ ;

```

- Complexité : Dans le pire cas (tous les sommets de G ont été enlevés, et seulement deux par itération (un point d'entrée et un point de sortie)), il y a $\left\lceil \frac{n}{2} \right\rceil$ pas d'itération.

Le k -ième pas d'itération nécessite un accès à `point_entrée`, et un à `point_sortie`, sur un sous-graphe comportant $n - 2(k - 1)$ sommets, soit :

- $n - 2k + 2$ accès `lst_pred` ;
- $n - 2k + 2$ accès `lst_pred`.

Au total, il y a donc au pire

$$\sum_{k=1}^{\left\lceil \frac{n}{2} \right\rceil} (n - 2k + 2)$$

accès `lst_pred`, et de même pour `lst_succ`.

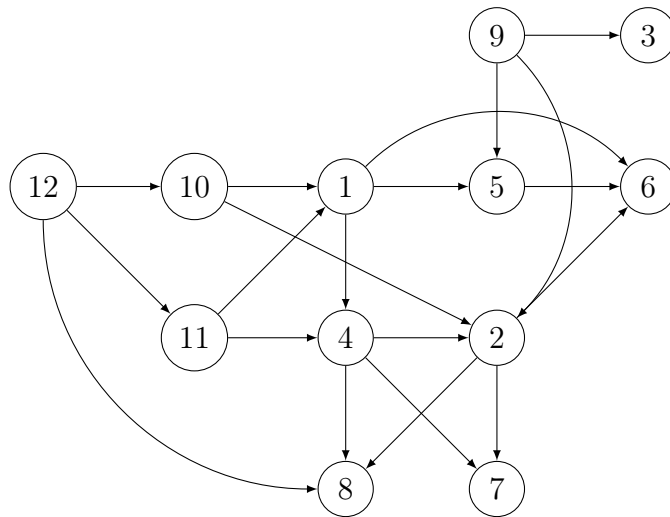
On a :

$$\sum_{k=1}^{\left\lceil \frac{n}{2} \right\rceil} (n - 2k + 2) = \begin{cases} \frac{n(n+2)}{4} & \text{si } n \equiv 0 \pmod{2} \\ \frac{(n+1)^2}{4} & \text{sinon} \end{cases}$$

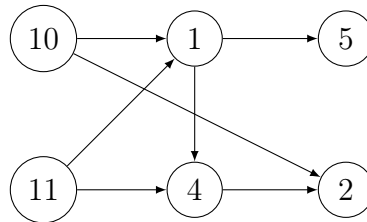
Donc $\mathcal{O}(n^2)$ accès `lst_pred` et `lst_succ`.

6.1.2 Exemple :

Soit le graphe :



Après la première étape, il devient :



Après la deuxième étape, il devient :



Et après la dernière étape, on obtient le graphe vide.
Donc le graphe initial était sans circuit.

6.2 Composantes fortement connexes

6.2.1 Définition

Soit $G = (X, \Gamma)$ un graphe.

On considère la relation binaire sur X suivante :

$$x\mathcal{R}y \equiv \begin{cases} x \in \Gamma^*(y) \\ y \in \Gamma^*(x) \end{cases}$$

avec $\Gamma^*(x) = \Gamma^+(x) \cup \{x\}$.

C'est une relation d'équivalence dont les classes s'appellent *composantes fortement connexes* (cfc en abrégé).

6.2.2 Proposition (6.2)

Deux sommets appartiennent à la même composante fortement connexe si et seulement si ils sont situés sur un même circuit.

6.2.3 Proposition (6.3)

Si deux sommets x et y appartiennent à une même composante fortement connexe C , alors tout chemin de x à y est entièrement inclus dans C .

6.2.4 Définition (6.2)

Un graphe est dit *fortement connexe* s'il possède une seule composante fortement connexe.

On remarque que si un graphe $G = (X, \Gamma)$ possède :

- au plus n composantes fortement connexes : cas obtenu si et seulement si G est sans circuit. Chaque composante fortement connexe est alors réduite à un seul sommet.
- au moins une composante fortement connexe : cas d'un graphe fortement connexe (tous les sommets sont sur un même circuit).

6.2.5 Définition (6.3)

Soit $\mathcal{R}$ une relation d'équivalence sur X .

Le graphe réduit selon les classes de $\mathcal{R}$ est défini par :

$$G_{\mathcal{R}} = (X/\mathcal{R}, \Gamma_{\mathcal{R}})$$

Où $X/\mathcal{R}$ est l'ensemble des classes de la relation $\mathcal{R}$ (ensemble quotient), et :

$$(C_i, C_j) \in \Gamma_{\mathcal{R}} \Leftrightarrow \exists x \in C_i, \exists y \in C_j \mid \begin{array}{l} (x, y) \in \Gamma \\ C_i \neq C_j \end{array}$$

Si on prend pour $\mathcal{R}$ la relation qui a servi à définir les composantes fortement connexes, on obtient le *graphe réduit selon les composantes fortement connexes*.

6.2.6 Proposition (6.4)

Le graphe réduit selon les composantes fortement connexes est sans circuit.

6.2.7 Définition (6.4)

On appelle *composante fortement connexe terminale* de G un point de sortie du graphe réduit.

6.2.8 Proposition (6.5)

Soit C un sous-ensemble de sommets tel que

$$\forall x, y \in C, \begin{cases} x \in \Gamma^*(y) \\ y \in \Gamma^*(x) \end{cases}$$

Si C n'a pas d'arc sortant, alors C est une composante fortement connexe terminale.

6.2.9 Algorithme de FOULKES

Cet algorithme s'applique à la fermeture transitive G^+ de G (on la suppose déjà calculée).

Il se base sur la définition :

$$x\mathcal{R}y \Leftrightarrow \begin{cases} (x, y) \in \Gamma^+ \\ (y, x) \in \Gamma^+ \end{cases}$$

- Principe : l'algorithme s'appuie sur trois considérations :
 - un sommet appartient à sa propre classe ;
 - Un sommet ne peut appartenir qu'à une seule classe ;
 - Si un sommet n'est pas son propre descendant, il est seul dans sa classe (il n'est pas situé sur un circuit).

Les sommets sont examinés séquentiellement.

Soit x le sommet courant.

Trois cas sont à distinguer concernant x :

- (1) x a déjà été rangé comme élément de la composante fortement connexe d'un sommet y examiné avant lui (*i.e* $y < x$). Dans ce cas, $\text{cl}(x) = \text{cl}(y)$ (déjà calculé).
- (2) x n'est pas encore classé, et n'est pas son propre descendant (*i.e* $(x, x) \notin \Gamma^+$) : dans ce cas, $\text{cl}(x) = \{x\}$.
- (3) x n'est pas encore classé, et $(x, x) \in \Gamma^+$.

Dans ce cas, les éléments de $\text{cl}(x)$, outre x lui-même, sont à rechercher parmi les y tels que $y > x$ (en effet, s'il existait $y < x$ avec $y \in \text{cl}(x)$, alors x aurait été classé lors de l'examen de y).

Algorithm 9: Algorithme de FOULKES

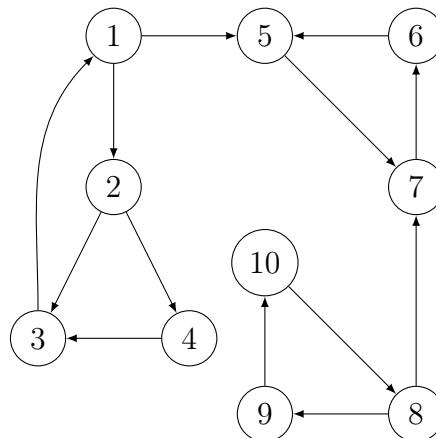
```

1  ens[sommets] sommet::cl;
2  Procedure foulkes(graphe  $G^+$ ):
3      var ens[sommets] pris  $\leftarrow \emptyset$ ;
4      var sommet  $x, y$ ;
5      for  $x = 1$  à  $n$  do
6          if  $x \notin$  pris then
7               $x.d \leftarrow \{x\}$ ;
8              if  $G^+.validarc(x, x)$  then
9                  //  $x$  est sur un circuit ; recherche des sommets de
                     sa classe
10                 for  $y = x + 1$  à  $n$  do
11                     if  $G^+.validarc(x, y)$  et  $G^+.validarc(y, x)$  then
12                          $x.cl \leftarrow x.cl \cup \{y\}$ ;
13                 for tout  $y$  de  $x.cl$  do
14                      $y.cl \leftarrow x.cl$ ;
15                 pris  $\leftarrow$  pris  $\cup x.cl$ ;

```

Complexité :

- Construction de la fermeture transitive : $\mathcal{O}(n^3)$;
- Nombre d'accès validarc : inférieur ou égal à n^2 .

6.2.10 Exemple

• $x = 1$

1.cl $\leftarrow \{1\}$

1.cl $\leftarrow \{1, 2, 3, 4\}$

2.cl $\leftarrow \{1, 2, 3, 4\}$

3.cl $\leftarrow \{1, 2, 3, 4\}$

4.cl $\leftarrow \{1, 2, 3, 4\}$

$\text{pris} \leftarrow \{1, 2, 3, 4\}.$

- $x = 2$: rien
- $x = 3$: rien
- $x = 4$: rien

- $x = 5$

$5.\text{cl} \leftarrow \{5\}$

$5.\text{cl} \leftarrow \{5, 6, 7\}$

$6.\text{cl} \leftarrow \{5, 6, 7\}$

$7.\text{cl} \leftarrow \{5, 6, 7\}$

$\text{pris} \leftarrow \llbracket 1 ; 7 \rrbracket$

- $x = 6$: rien
- $x = 7$: rien
- $x = 8$

$8.\text{cl} \leftarrow \{8\}$

$8.\text{cl} \leftarrow \{8, 9, 10\}$

$9.\text{cl} \leftarrow \{8, 9, 10\}$

$10.\text{cl} \leftarrow \{8, 9, 10\}$

$\text{pris} \leftarrow \llbracket 1 ; 10 \rrbracket$

- $x = 9$: rien
- $x = 10$: rien

On a trois composantes fortement connexes :

$\{1, 2, 3, 4\}$;

$\{5, 6, 7\}$

$\{8, 9, 10\}$

7 Chemins de valeur optimale

7.1 Définition et problèmes posés

7.1.1 Définition (7.1)

On appelle *graphe valué* un doublet $G = (X, v)$, où v est une fonction définie sur X^2 et à valeurs dans un ensemble E .

On va supposer que l'ensemble E possède une loi de composition $\otimes$ associative, et munie d'un élément neutre e ainsi que d'un élément absorbant α .

L'élément absorbant α permet de caractériser l'ensemble Γ des arcs :

$$(x, y) \in \Gamma \Leftrightarrow v(x, y) \neq \alpha$$

On a, par ailleurs :

$$\forall w \in E, w \otimes e = e \otimes w = w$$

$$\forall w \in E, w \otimes \alpha = \alpha \otimes w = \alpha$$

Et :

$$\forall w_1, w_2 \in E, w_1 \otimes w_2 = \alpha \Rightarrow \begin{cases} w_1 = \alpha \\ \text{ou} \\ w_2 = \alpha \end{cases}$$

7.1.2 Exemples

E	$\otimes$	e	α	
$B = \{\text{vrai}, \text{faux}\}$	$\wedge$	vrai	faux	valeurs d'existence
$\mathbb{R}$	$+$	0	$+\infty$	valeurs additives
$[0 ; 1]$	$\times$	1	0	probabilités

L'associativité de $\otimes$ permet de prolonger v à l'ensemble des suites de sommets de longueur ≥ 2 :

$$v[x_1, \dots, x_p] = \bigotimes_{j=1}^{p-1} v(x_j, x_{j+1})$$

7.1.3 Proposition (7.1)

Soit $[x_1, \dots, x_p]$ (avec $p \geq 2$) une suite de sommets de X .

Cette suite définit un chemin de G si et seulement si $v[x_1, \dots, x_p] \neq \alpha$.

Notre problème est l'étude des chemins de valeur optimale.

Il faut définir un critère de comparaison sur les valeurs des chemins.

E doit être muni d'une relation d'ordre totale, notée $\preceq$, telle que :

(1) $\preceq$ soit compatible avec $\otimes$, i.e

$$w_1 \preceq w_2 \Rightarrow \begin{cases} w_1 \otimes w_3 \preceq w_2 \otimes w_3 \\ w_3 \otimes w_1 \preceq w_3 \otimes w_2 \end{cases}$$

(2) α est maximal pour $\preceq$:

$$\forall w \in E, w \preceq \alpha$$

Soient deux suites

$$\begin{aligned} u_1 &= [x, y_1, \dots, y_k, y] \\ u_2 &= [x, z_1, \dots, z_p, y] \end{aligned}$$

On dira que u_1 est *meilleur* que u_2 si et seulement si

$$v(u_1) \preccurlyeq v(u_2)$$

Exemple :

E	$\preccurlyeq$
B	vrai < faux
$\mathbb{R}$	$\leq$
proba	$\geq$

Trois variantes du problème :

- (x -optimal- y) : Soient x, y deux sommets donnés de $G = (X, \Gamma, v)$.
On veut déterminer un chemin $u = (x * y)$ de G de valeur optimale parmi tous les chemins de x à y dans G .
- (x -optimal) ou (y -optimal) : problème optimal- y pour tout $y \in X$ (respectivement x -optimal- y pour tout $x \in X$).

Dans la suite, on prendra :

$$E \equiv \mathbb{R} \quad \otimes \equiv + \quad \preccurlyeq \equiv \leq$$

Valeurs additives minimales.

7.2 Chemins de valeur additive minimale issus d'un sommet donné (problème x -optimal)

On supposera les sommets numérotés de 1 à n , et que le sommet de départ est le sommet 1.

7.2.1 Existence

- Propriété (7.2)

Le problème 1-minimal admet des solutions si et seulement si le sous-graphe

$$G_1 = (\Gamma^*(1), \Gamma_1)$$

ne contient pas de circuit de valeur négative.

- Définition (7.2) : Un circuit de valeur strictement négative s'appelle un *circuit absorbant*.

7.2.2 Algorithme de DIJKSTRA

Considère des graphes valués où les arcs sont de valeur ≥ 0 .

$G = (X, \Gamma, v)$ est un graphe valué, avec

$$\forall \gamma \in \Gamma, v(\gamma) \geq 0$$

Donc G ne possède aucun circuit absorbant.

C'est un algorithme glouton exact (il ne fait pas de retour en arrière (*backtracking*)) de complexité polynomiale.

- Principe :

Entrée :

- Graphe orienté pondéré par des réels ≥ 0 ;
- un sommet source.

On construit progressivement un sous-graphe dans lequel les différents sommets sont classés par ordre croissant de leur distance minimale au sommet de départ.

- Initialisation : la distance du sommet de départ à chaque sommet est $+\infty$ sauf pour le sommet de départ lui-même (zéro).

Le sous-graphe de départ est vide.

- Au cours d'une itération :

- On choisit *en dehors du sous-graphe* un sommet de distance minimale, et on l'ajoute au sous-graphe.

Puis on met à jour les distances des successeurs de x comme suit : la nouvelle distance de y est le minimum entre la distance existante et celle obtenue en ajoutant le poids de l'arc (x, y) à la distance de x .

$$\text{dist}(y) \leftarrow \min(\text{dist}(y), \text{dist}(x) + v(x, y))$$

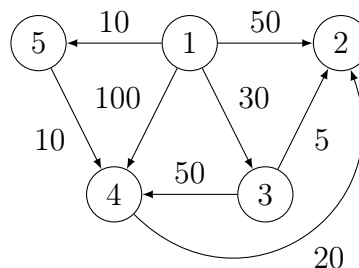
Algorithm 10: DIJKSTRA

```

1 Procédure dijkstra(graphe  $G$ , sommet source):
2   var ens[sommet]  $Q$ ; ent alt;
3   créer( $Q$ );
4   for chaque sommet  $u$  de  $G$  do
5      $\text{dist}[u] \leftarrow +\infty$ ;
6      $\text{pred}[u] \leftarrow \text{indéfini}$ ;
7     // sommet précédent dans le chemin optimal
8     ajouter( $u$ ,  $Q$ )
9    $\text{dist}[\text{source}] \leftarrow 0$ ;
10  while non vide( $Q$ ) do
11     $x \leftarrow$  sommet de  $Q$  avec distance minimale;
12    // source va être choisie en première
13    supprimer( $x$ ,  $Q$ );
14    for chaque successeur  $y$  de  $x$  présent dans  $Q$  do
15       $\text{alt} \leftarrow \text{dist}[x] + v(x, y)$ ;
16      if  $\text{alt} < \text{dist}[y]$  then
17         $\text{dist}[y] \leftarrow \text{alt}$ ;
18         $\text{pred}[y] \leftarrow x$ ;

```

- Exemple : soit le graphe suivant :



Source : sommet 1.

Initialisation :

	1	2	3	4	5
dist	0	∞	∞	∞	∞
pred	nil	nil	nil	nil	nil

$Q = \llbracket 1 ; 5 \rrbracket$.

Étape 1 :

$x \leftarrow 1$, $Q = \llbracket 2 ; 5 \rrbracket$.

	1	2	3	4	5
dist	0	50	30	100	10
pred	nil	1	1	1	1

Étape 2 :

$x \leftarrow 5$; $Q = \llbracket 2 ; 4 \rrbracket$.

	1	2	3	4	5
dist	0	50	30	20	10
pred	nil	1	1	5	1

Étape 3 :

$x \leftarrow 4$; $Q = \llbracket 2 ; 3 \rrbracket$.

	1	2	3	4	5
dist	0	40	30	20	10
pred	nil	4	1	5	1

Étape 4 :

$x \leftarrow 3$; $Q = \{2\}$.

	1	2	3	4	5
dist	0	35	30	20	10
pred	nil	3	1	5	1

Étape 5 :

$x \leftarrow 2$; $Q = \emptyset$.

Il n'y a plus de successeur de 2 encore dans Q (qui est vide), l'algorithme s'arrête donc.

Meilleur chemin de 1 à 4 ?

coût : 20 ; $1 \rightarrow 5 \rightarrow 4$.

7.2.3 Algorithme de BELLMAN–FORD

Contrairement à l'algorithme de DIJKSTRA, cet algorithme autorise la présence de poids négatifs et permet de détecter l'existence d'un éventuel circuit absorbant (*i.e* de poids total < 0 accessible depuis le sommet source).

La complexité de cet algorithme est en $\mathcal{O}(|S| \cdot |A|)$, avec $|S|$ le nombre de sommets, et $|A|$ le nombre d'arcs.

C'est un algorithme de type *programmation dynamique*.

Il retourne les distances d'un sommet source donné à tous les autres sommets.

Les sous-problèmes sont :

- $d[t, k]$ la distance du sommet source s au sommet t par un chemin qui contient au plus k arcs.

On a :

$$\begin{cases} d[t, 0] = +\infty & \text{si } t \neq s \\ d[s, 0] = 0 \\ d[t, k] = \min \left(d[t, k-1], \min_{\text{arc}(u,t)} (d[u, k-1] + v(u, t)) \right) \end{cases}$$

L'algorithme calcule les valeurs $d[t, k]$ par valeurs de k croissantes.

La valeur maximale, pour k , à considérer, est $|S| - 1$.

La distance optimale de s à t est $d[t, |S| - 1]$.

- Première version, qui utilise directement la relation de récurrence précédente :

Algorithm 11: BELLMAN-FORD, première version

```

1 Procedure bellman-ford(graphe  $G$ , sommet  $s$ ):
2   var ent  $k$ ;
3   var sommet  $u$ ,  $t$ ;
4   for chaque sommet  $u$  de  $X$  do
5      $d[u, 0] \leftarrow +\infty$ ;
6    $d[s, 0] \leftarrow 0$ ;
   //  $n = |X|$ 
7   for  $k = 1$  à  $n - 1$  do
8     for chaque sommet  $t$  de  $X$  do
9        $d' \leftarrow d[t, k - 1]$ ;
10      for chaque arc  $(u, t)$  de  $\Gamma$  do
11         $d' \leftarrow \min(d', d[u, k - 1] + v(u, t))$ ;
12       $d[t, k] \leftarrow d'$ ;
```

Algorithm 12: BELLMAN-FORD, seconde version

```

1 Procedure bellman-ford(graphe  $G$ , sommet  $s$ ):
2   var ent  $k$ ;
3   var sommet  $u$ ;
4   for chaque sommet  $u$  de  $X$  do
5      $\text{dist}[u] \leftarrow +\infty$ ;
6      $\text{pred}[u] \leftarrow \text{nil}$ ;
7    $\text{dist}[s] \leftarrow 0$ ;
8   for  $k = 1$  à  $n - 1$  do
9     for chaque arc  $(u, t)$  de  $\Gamma$  do
10      if  $\text{dist}[u] + v(u, t) < \text{dist}[t]$  then
11         $\text{dist}[t] \leftarrow \text{dist}[u] + v(u, t)$ ;
12         $\text{pred}[t] \leftarrow u$ ;
```

On peut détecter la présence d'un circuit absorbant en faisant un tour de boucle supplémentaire. Il y a un tel circuit si ce tour de boucle fait diminuer l'un des $\text{dist}[t]$.

À la fin de l'algorithme 12, on fait :

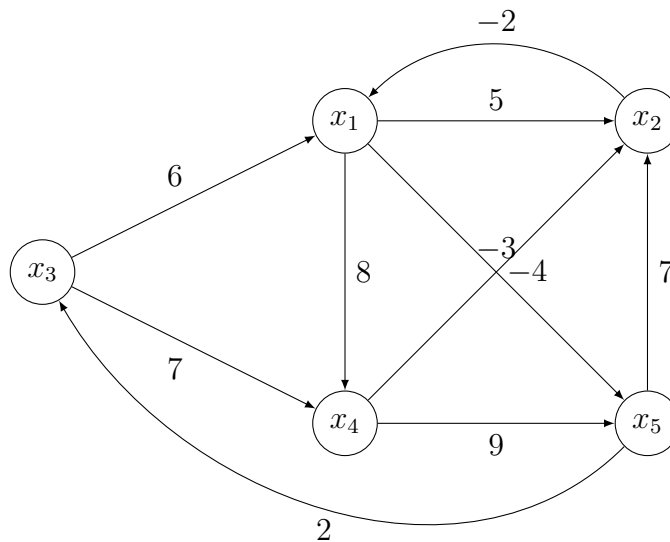
Algorithm 13: Détection de circuit absorbant

```

1 for chaque arc  $(u, t)$  de  $\Gamma$  do
2   if  $\text{dist}[u] + v(u, t) < \text{dist}[t]$  then
3     écrire("il existe un circuit absorbant");

```

- Exemple : soit le graphe suivant :



dist :

X	x_1	x_2	x_3	x_4	x_5
initialisation	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$
étape 1	6	$+\infty$	0	7	$+\infty$
étape 2	6	4	0	7	2
étape 3	2	4	0	7	2
étape 4	2	4	0	7	-2

pred :

X	x_1	x_2	x_3	x_4	x_5
initialisation	/	/	/	/	/
étape 1	x_3	/	/	x_3	/
étape 2	x_3	x_4	/	x_3	x_1
étape 3	x_2	x_4	/	x_3	x_1
étape 4	x_2	x_4	/	x_3	x_1

Un tour de boucle supplémentaire ne change rien, il n'y a donc pas de circuit absorbant.

Meilleur chemin de x_3 à x_5 : coût = -2 ;

$$x_3 \rightarrow x_4 \rightarrow x_2 \rightarrow x_1 \rightarrow x_5$$

Si on change le poids de l'arc (x_5, x_2) à 4 dans le graphe, on obtient un circuit de poids négatif au niveau du nœud x_2 quand on teste l'arc (x_5, x_2) .
Il s'agit du circuit $[x_1, x_5, x_2]$.

7.3 Valeurs optimales de tout sommet à tout sommet

Idée : adapter l'algorithme de ROY–WARSHALL (qui calcule la fermeture transitive).
L'algorithme modifié est l'algorithme de FLOYD–WARSHALL.
On a les trois lemmes suivants :

7.3.1 Lemme 1

A l'issue de l'algorithme,

$$\forall z, y \in X, y.R[z] \neq \text{nil} \Leftrightarrow (y, z) \in \Gamma^+$$

Où $y.R[z]$ est un successeur de y sur un chemin élémentaire de y à z .

7.3.2 Lemme 2

Si G n'a pas de circuit absorbant, alors pour tout couple $(y, z) \in \Gamma^+$, on a, à l'issue de l'algorithme, $y.R[z]$ est le successeur de y sur un chemin élémentaire de y à z .

7.3.3 Lemme 3

Si G n'a pas de circuit absorbant, alors pour tout couple $(y, z) \in \Gamma^+$, tout chemin de valeur minimale de y à z est élémentaire.

Donc, si G n'a pas de circuit absorbant, les tables de routage R permettent d'obtenir les tracés de chemins de valeur minimale entre tout couple de sommet

```
table[sommet, sommet] sommet::R
```

7.3.4 Rappel de l'algorithme de ROY–WARSHALL avec routage

Algorithm 14: Rappel de l'algorithme de ROY–WARSHALL avec routage

```

1 for tout  $x$  de  $G$ .sommets do
2   for tout  $y$  de  $G$ .sommets do
3     if  $\text{res.existeArc}(y, x)$  then
4       for tout  $z$  de  $G$ .sommets do
5         if  $\text{res.existeArc}(x, z)$  then
6           if non  $\text{existeArc}(y, z)$  then
7              $\text{res.ajouterArc}(y, z)$ ;
8              $y.R[z] \leftarrow y.R[x]$ ;

```

7.3.5 Algorithme de FLOYD–WARSHALL**Algorithm 15:** Algorithme de FLOYD–WARSHALL

```

1 for tout  $x$  de  $G$ .sommets do
2   for tout  $y$  de  $G$ .sommets do
3     if  $\text{res.existeArc}(y, x)$  then
4        $\text{vyx} \leftarrow \text{res.v}(y, x)$ ;
5       for tout  $z$  de  $G$ .sommets do
6         if  $\text{res.existeArc}(x, z)$  then
7            $\text{vxy} \leftarrow \text{res.v}(x, z)$ ;
8           if non  $\text{existeArc}(y, z)$  then
9              $\text{res.ajouterArc}(y, z)$ ;
10             $y.R[z] \leftarrow y.R[x]$ ;
11         else
12            $\text{vyz} \leftarrow \text{res.v}(y, z)$ ;
13           if  $\text{vyz} > \text{vyx} + \text{vxz}$  then
14              $\text{res.modifierArc}(y, z, \text{vyx} + \text{vxy})$ ;
15              $y.R[z] \leftarrow y.R[x]$ ;

```

Complexité : en $\mathcal{O}(n^3)$ (cf ROY–WARSHALL).